SIMATS ENGINEERING

CO2_AT2 Reverse Engineering Task

Course Name:	Software Testing	**Course Code:**	CSA37
Student Name:	Ganesh Karthick M	**Reg. No.:**	192321138
Section / Batch:		**Date:**	24-10-2026

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

QUESTIONS: 10

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5
4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5

Sl. No	Questions	Bloom's Level
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5
9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5

Code of Conduct:

I Ganesh Karthick M / 192321138 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

RUBRICS FOR CALCULATION:3

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Annexure A – Reverse Engineering Task: Answers

No source code or raw data accompanies the questions, so each answer states the observed artefacts it assumes (sample logs, traces, profiles). The reasoning and reconstructed test suites follow from those assumptions.

Question 1: ATM Withdrawal Module

Assumed observations (input → output only): 500 → dispensed; 1000 → dispensed; 0 → "Invalid amount"; 15000 → "Exceeds per-transaction limit"; 500 with balance 200 → "Insufficient balance".

1. Decomposition into functional components

Component	Responsibility	Observable evidence
Card and PIN check	Authenticate before any withdrawal	Not covered by the observed cases
Amount validation	Numeric, positive, multiple of 100, 100 to 10,000	"Invalid amount", "Exceeds limit"
Balance check	Amount ≤ available balance	"Insufficient balance"
Daily limit check	Total withdrawn today ≤ 40,000	No evidence (gap)
Cash dispenser	Notes available for the amount	No evidence (gap)
Receipt and ledger update	Print receipt, debit account	Implicit

2. Equivalence classes and boundary conditions (inferred rules: min 100, max 10,000 per transaction, multiples of 100)

Input	Class	Representative	Valid?	Seen in observed tests?
Amount	E1 non-numeric / empty	abc	No	No
Amount	E2 zero or negative	0, −500	No	Yes (0 only)
Amount	E3 1 to 99	50	No	No
Amount	E4 100 to 10,000, multiple of 100	500	Yes	Yes
Amount	E5 100 to 10,000, not a multiple of 100	150	No	No
Amount	E6 above 10,000	15000	No	Yes
Balance	B1 balance ≥ amount / B2 balance < amount	1000 / 200	Yes / No	Yes / Yes
Daily total	D1 within 40,000 / D2 exceeds	—	Yes / No	No / No

Boundaries: amount 99 | 100 | 200 and 9,900 | 10,000 | 10,100; balance equal to amount (exactly 0 left); daily total 40,000 | 40,100.

3. Inferred techniques and adequacy

Existing cases are one typical valid value, one zero, one very large value, and one insufficient-balance case. This pattern suggests **basic equivalence partitioning at class level plus error guessing**; there is no sign of boundary value analysis (no 99, 100, 10,000 or 10,100) and no combination testing.

Measure	Result
Amount classes covered	3 of 6 (50%)
Balance classes covered	2 of 2 (100%)
Daily-limit classes covered	0 of 2 (0%)
Boundary values tested	0 of 8 (0%)

Measure	Result
Verdict	Inadequate: the boundaries where off-by-one defects occur are untested

4. Reconstructed (improved) test suite

TC	Amount	Balance	Daily used	Technique	Expected result
1	abc	5000	0	ECP E1	Invalid amount
2	0	5000	0	ECP E2	Invalid amount
3	−500	5000	0	ECP E2	Invalid amount
4	99	5000	0	BVA	Invalid amount
5	100	5000	0	BVA min	Dispensed 100
6	200	5000	0	BVA min+	Dispensed 200
7	150	5000	0	ECP E5	Invalid: must be multiple of 100
8	500	5000	0	ECP E4	Dispensed 500
9	9900	20000	0	BVA max−	Dispensed 9,900
10	10000	20000	0	BVA max	Dispensed 10,000
11	10100	20000	0	BVA max+	Exceeds per-transaction limit
12	500	500	0	Balance boundary	Dispensed; balance 0
13	500	400	0	Balance − 1 step	Insufficient balance
14	10000	50000	30000	Daily boundary	Dispensed (total 40,000)
15	100	50000	40000	Daily boundary +	Daily limit exceeded
16	500	5000	0	Dispenser empty	Service unavailable, no debit
17	500	5000	0	Wrong PIN x3	Card retained, no cash

Interpretation: 17 cases cover every inferred class and every boundary of the three limits; the original 5 cases covered only 3 of the 6 amount classes and none of the boundaries.

Question 2: Reverse Engineering the White Box Approach from Traces and Code

Assumed code snippet (n, array a, limit):

```
1: sum = 0; i = 0
2: if (n <= 0) return -1
4: while (i < n) {
5:   if (a[i] < 0) { i++; continue }
7:   if (sum + a[i] > limit) break
8:   sum += a[i]; i++ }
9: if (sum == 0) return 0
11: return sum
```

Assumed traces in the log: T1: n = 0; T2: a = [4, 6], limit 20 (sum 10); T3: a = [−3, 5], limit 20 (sum 5).

1. Control flow graph

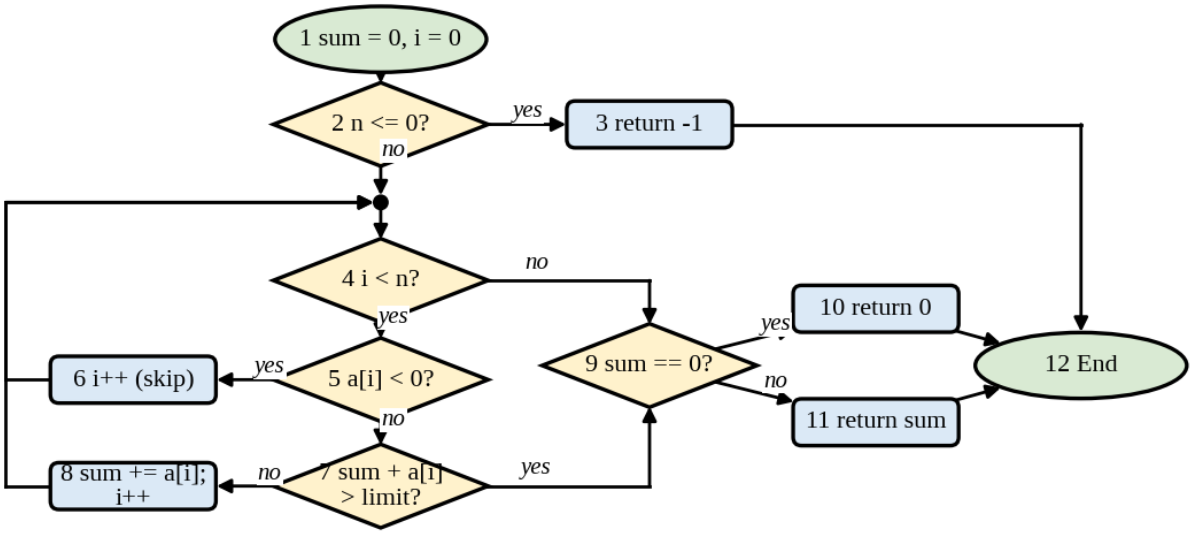


Figure 1: Control flow graph of the module (node numbers match the code lines)

2. Decision points, loops and independent paths

Decision points: node 2 (n ≤ 0), node 4 (loop condition i < n), node 5 (negative element), node 7 (limit exceeded), node 9 (sum = 0). **Loop:** the while loop 4→5…→4. **Cyclomatic complexity:** V(G) = E − N + 2 = 16 − 12 + 2 = 6 (also 5 decisions + 1 = 6), so 6 independent paths are needed.

Path	Node sequence	Scenario	Test input	Expected
P1	1-2-3-12	n <= 0	n = 0	−1
P2	1-2-4-5-7-8-4-9-11-12	Positive element within limit	a = [5], limit 10	5
P3	1-2-4-5-6-4-9-10-12	Only negative element, skipped	a = [−2], limit 10	0
P4	1-2-4-5-7-9-10-12	First element exceeds limit (break)	a = [15], limit 10	0
P5	1-2-4-5-7-8-4-9-10-12	Element 0 within limit (sum stays 0)	a = [0], limit 10	0
P6	1-2-4-5-7-8-4-5-7-9-11-12	Positive element, then limit exceeded	a = [6, 7], limit 10	6

The structural path 1-2-4-9… (loop never entered although n > 0) is infeasible because n > 0 guarantees one iteration, which is why every feasible basis path above enters the loop except P1.

3. Adequacy of the existing traces

Criterion	Covered by T1 to T3	Percentage
Statement (12 nodes)	11 (node 10 'return 0' never executed)	91.7%
Branch (10 decision outcomes)	8 (node 7 'true' and node 9 'true' never taken)	80%
Independent paths (6)	3 (P1, P2 type, skip type); P4, P5, P6 missing	50%

The traces look like typical-value testing driven by statement coverage: the break on the limit and the "total is zero" results are never exercised.

4. Reconstructed path testing strategy

Test	Input	Covers	Expected
TC1	n = 0	P1	−1
TC2	a=[5], limit 10	P2	5
TC3	a=[−2], limit 10	P3	0
TC4	a=[15], limit 10	P4	0
TC5	a=[0], limit 10	P5	0
TC6	a=[6, 7], limit 10	P6	6
TC7	a=[10], limit 10	Boundary: 0 + 10 > 10 is false	10
TC8	a=[6, 4], limit 10	Boundary: 6 + 4 > 10 is false	10
TC9	a=[−2, 4], limit 10	Skip then add (mixed loop)	4

Result: TC1 to TC6 give 100% statement, 100% branch and all 6 independent paths; TC7 to TC9 add boundary and loop-mix cases. **Interpretation:** a complete basis-path suite is the smallest set that guarantees every decision outcome is exercised.

Question 3: Web Login and Transaction System – Reverse Engineering from Defect Logs

Assumed defect log (8 defects):

ID	Defect	Pattern	Likely missed technique
D1	Login fails when password length is exactly 8 (minimum)	Boundary	BVA
D2	Transfer of 0.00 is accepted	Boundary / invalid class	BVA, ECP
D7	Transfer of exactly 1,00,000 (maximum) rejected	Boundary	BVA
D3	Session still valid after logout (back button)	Path / state	Path testing
D6	Account not locked after 3 failures when OTP also fails	Branch missing counter	Path / branch coverage
D4	Special characters in username cause server error	Input validation	Error guessing, ECP
D5	Wrong message when amount ≤ daily limit but > balance	Condition combination	Decision table
D8	Duplicate transaction on double click	Concurrency	Stress / state testing

1. Pattern analysis – what techniques were probably used

Category	Defects	Share of 8	Inference
Boundary-related	D1, D2, D7	37.5%	BVA was not applied
Logic and path	D3, D6	25%	Only statement-level white-box testing
Input validation / security	D4	12.5%	Error guessing absent
Combination of conditions	D5	12.5%	No decision-table testing
Concurrency	D8	12.5%	No stress testing

Defects escaped where values sit on edges and where conditions combine, while typical valid/invalid inputs were evidently found. This points to **basic ECP with typical values, ad-hoc exploratory checks and statement coverage only** as the techniques likely used.

2. Decomposition and coverage gaps

Component	Testable behaviour	Technique needed	Gap found
Login	Password length 8 to 16, lock after 3 failures	ECP, BVA, path	D1, D6
Session	Create, expire, destroy on logout	State / path	D3
Transaction input	Amount 1 to 1,00,000, balance, daily limit	ECP, BVA, decision table	D2, D5, D7
Data layer / input handling	Special characters, SQL strings	Error guessing	D4
Concurrency	Repeated submission	Stress, state	D8

3. Reconstructed testing strategy

Technique	Test design	Expected outcome
ECP (login)	Password length classes: <8 invalid, 8 to 16 valid, >16 invalid	Reject / accept / reject
BVA (login)	Lengths 7, 8, 9, 15, 16, 17	7 rejected; 8, 9, 15, 16 accepted; 17 rejected

Technique	Test design	Expected outcome
ECP + BVA (transfer)	Amounts 0, 1, 99,999, 1,00,000, 1,00,001 and a typical 5,000	0 and 1,00,001 rejected; others accepted
Decision table	Conditions: amount in limit, amount ≤ balance, within daily limit (8 rules)	Correct message for each rule
Path testing (login)	Basis paths: locked, wrong password, count to lock, wrong OTP, success	Counter increments on every failed factor
Session state testing	Login, logout, press Back, reuse token	Access denied after logout
Error guessing	Quotes, SQL fragments, very long strings in username	Handled; no server error
Stress / concurrency	Double click, 100 parallel submissions	One transaction only

Justification: boundary, path and condition-combination defects make up 75% of the log (6 of 8), and ECP, BVA, path testing and the decision table are exactly the techniques designed to find them; the remaining two (input validation, concurrency) are covered by error guessing and stress testing.

Question 4: Legacy Inventory System – Reverse Engineering from Transaction Logs

Assumed log extract:

Log	Stock	Transaction	Result	Alert
L1	50	Issue 10	Success, stock 40	None
L2	15	Issue 5	Success, stock 10	None
L3	12	Issue 3	Success, stock 9	LOW STOCK
L4	20	Issue 25	Fail: insufficient stock	—
L5	480	Receive 30	Fail: capacity exceeded (510)	—
L6	450	Receive 50	Success, stock 500	None
L7	30	Issue 0	Fail: invalid quantity	—

1. Inferred business logic and hidden boundaries

Rule	Evidence	Boundary
Low-stock alert when new stock < 10	L2 (10, no alert) vs L3 (9, alert)	10 / 9
Maximum capacity 500	L5 (510 fail) vs L6 (500 success)	500 / 501
Quantity must be > 0	L7 (0 rejected)	0 / 1
Cannot issue more than stock	L4 (25 > 20 fail)	stock / stock + 1

2. Equivalence partitions of the resulting stock

Partition	Range of new stock	Expected behaviour
P1	< 0	Rejected: insufficient stock
P2	0	Success; alert (out of stock)
P3	1 to 9	Success; low-stock alert
P4	10 to 500	Success; no alert
P5	> 500	Rejected: capacity exceeded

3. Functional units and gaps in the logs

Units: (1) quantity validation, (2) stock arithmetic (issue/receive), (3) capacity check, (4) alert generation, (5) transaction logging. **Gaps:** the logs never show an issue that leaves exactly 0, an issue of stock + 1, a receive to 501, a negative or fractional quantity, quantity 1 (the minimum), or whether the alert repeats.

4. Reconstructed black box test design

TC	Stock	Transaction	Targets	Expected
1	30	Issue 1	Minimum quantity	Success, 29, no alert
2	30	Issue 0	Quantity boundary	Fail: invalid quantity
3	30	Issue −5	Negative	Fail: invalid quantity
4	30	Issue 2.5	Non-integer	Fail: invalid quantity
5	20	Issue 20	Gap: result exactly 0	Success, 0, out-of-stock alert
6	20	Issue 21	Gap: stock + 1	Fail: insufficient stock

TC	Stock	Transaction	Targets	Expected
7	19	Issue 10	Result exactly 9	Success, 9, LOW STOCK
8	20	Issue 10	Result exactly 10	Success, 10, no alert
9	499	Receive 1	Result exactly 500	Success, 500
10	500	Receive 1	Result 501	Fail: capacity exceeded
11	9	Receive 1	Recovery from low stock	Success, 10, alert cleared
12	8	Issue 1 then Issue 1	Repeated alerts	Alert at 7 and at 6 (or once, per spec)

Justification: each test sits on a boundary inferred from the logs (9/10, 500/501, 0/1, stock/stock + 1) or fills a gap the logs never exercised, so the 12 cases cover all five partitions and all four rules.

Question 5: Library Module – Executable and Stress-Test Profile

Assumed profile (4 cores, 200 simulated users): CPU spike A at 10 to 15 s (85 to 100%) during catalogue search; CPU spike B at 40 to 50 s (95%) during overdue-fine calculation; memory rising steadily from 120 MB to 480 MB and not released after borrow operations.

1. Mapping spikes to likely logic

Profile symptom	Likely construct	White-box requirement
Spike A: CPU burst during search	Loop over all books (loop 3, O(n))	Loop boundary tests (0, 1, many books), match/no-match decisions
Spike B: CPU burst during fines	Loop over a member's loans, nested in return handling (loop 9)	Loop tests with 0, 1, many loans; fine rule decisions
Memory ramp after borrowing	Allocation in the issue path (node 7) without release	Repeat the borrow path many times (soak path test)
Sustained 100% CPU	Retry or polling loop in the reservation check	Termination condition tests

Quantifying spike A: 100,000 books × 5 µs per comparison = 0.5 s per search. With 200 concurrent searches that is 100 CPU-seconds of work per second, about 25 times what 4 cores can supply (100 / 4 = 25), which explains the saturation.

2. Conceptual control flow graph

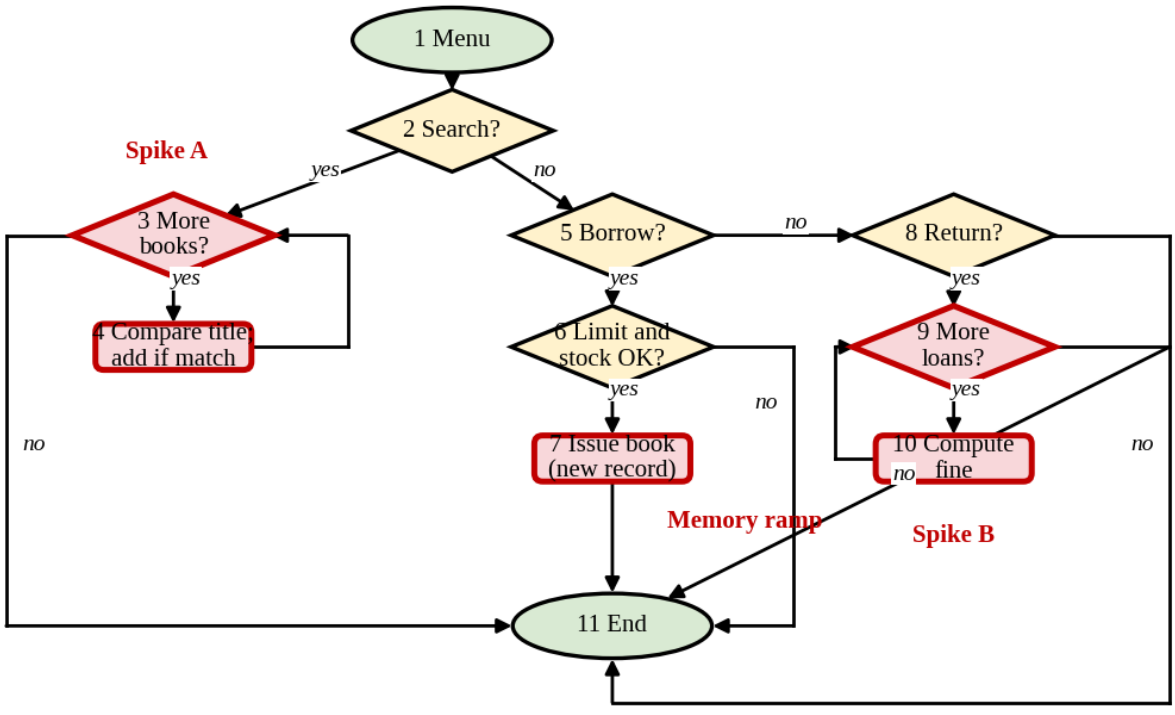


Figure 2: Conceptual CFG with resource-intensive nodes (red)

Cyclomatic complexity: E = 16, N = 11, so V(G) = 16 − 11 + 2 = 7 (6 decisions + 1).

3. Path testing strategy

Path	Nodes	Scenario	Stress variant
P1	1-2-3-11	Search, empty catalogue	—

Path	Nodes	Scenario	Stress variant
P2	1-2-3-4-3-11	Search, one or more books (loop)	100,000 books, 200 users (spike A)
P3	1-2-5-6-11	Borrow rejected (limit or stock)	—
P4	1-2-5-6-7-11	Borrow accepted	10,000 borrows in a row, watch memory (ramp)
P5	1-2-5-8-11	Neither borrow nor return	—
P6	1-2-5-8-9-11	Return, no loans	—
P7	1-2-5-8-9-10-9-11	Return with fine loop	Member with 500 loans (spike B)

4. Impact on system stability

Metric	Before (assumed)	Target after fixes and re-test
CPU during search	85 to 100%	< 70% (index or cache on title)
Memory growth over 1 h soak	+360 MB (120 to 480)	Flat within ±5%
p95 response time	> 5 s	< 2 s
Termination of retry loop	Not guaranteed	Bounded retries with error

Interpretation: directing path tests (P2, P4, P7) at the hot paths turns the profile into measurable stability criteria; the 7 paths also give full branch coverage of the module.

Question 6: Mobile Payment Gateway – State Machine Reconstruction

Assumed JSON observations: {"txnId":"T1","status":"PENDING"} followed by {"status":"SUCCESS"}; PENDING with no reply for 90 s followed by "TIMEOUT"; TIMEOUT followed by "SUCCESS" when a late bank confirmation arrives or by "REVERSAL" when no debit is confirmed; SUCCESS followed by "REVERSAL" on a refund request.

1. Reconstructed state machine

From	Trigger [guard]	To
(start)	initiate payment	Pending
Pending	bank confirms	Success
Pending	no response > 90 s	Timeout
Timeout	late bank confirmation	Success
Timeout	reconciliation finds no confirmed debit	Reversal
Success	refund request [within 7 days, refund ≤ original amount]	Reversal

Data constraints inferred: txnId is unique; refund amount ≤ original; refund within 7 days; a repeated confirmation for the same txnId is idempotent.

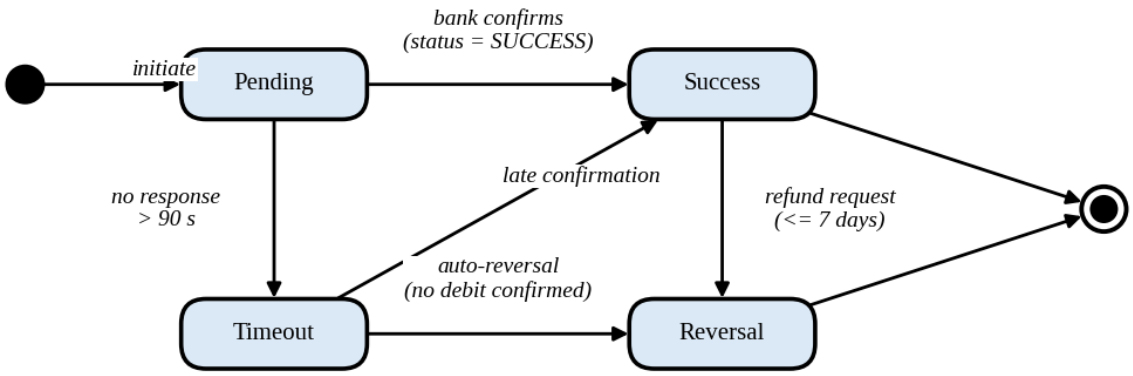


Figure 3: State-transition diagram of the payment gateway

2. Implicit technique

The responses depend on the *current state and the event*, not just the input value, so the developers used (or should be tested by) **State Transition Testing**: states, events, guards and actions, with transition (0-switch) coverage and selected sequences (1-switch).

3. State × event matrix (legal = target state, illegal = X)

State	E1 bank confirms	E2 90 s elapsed	E3 refund request	E4 auto-reversal
Pending	Success	Timeout	X	X
Timeout	Success	X	X	Reversal
Success	X (ignored, idempotent)	X	Reversal	X
Reversal	X	X	X	X

4 states × 4 events = 16 cells: 5 legal and 11 illegal.

4. State-based test suite

TC	Start state	Event / guard	Expected next state / response
1	(start)	initiate	Pending
2	Pending	bank confirms	Success
3	Pending	90 s elapsed	Timeout
4	Timeout	late confirmation	Success
5	Timeout	reconciliation, no debit	Reversal
6	Success	refund, amount = original, day 7	Reversal
7	Success	refund, amount = original + 1	Rejected, stays Success
8	Success	refund on day 8	Rejected, stays Success
9	Pending	refund request	Illegal: rejected, stays Pending
10	Pending	auto-reversal	Illegal: rejected
11	Timeout	90 s elapsed again	Illegal: no change
12	Success	bank confirms again	Ignored (idempotent)
13	Reversal	bank confirms	Illegal: stays Reversal
14	Reversal	refund request	Illegal: stays Reversal
15	Pending, Timeout, Success	sequence of E2, E1, E3 (1-switch)	Timeout, Success, Reversal

Coverage: TC1 to TC6 cover all 6 legal transitions (including start); TC7 to TC14 cover the guards and 8 of the 11 illegal cells, and extra negative cases can be added for the remaining three; TC15 checks a multi-step sequence. **Interpretation:** value-based tests cannot find illegal transitions such as refunding a pending payment, so a state-based suite is required.

Question 7: Encrypted Communication Module – Fuzzing and BVA

Assumed fuzzing results (input fields: message, recipientId, keySize):

Fuzz input	Observed output	Inference
message length 0	ERR_EMPTY	Minimum length 1
message length 256	Encrypted output	Maximum valid length
message length 257	ERR_LEN	Upper limit 256
message of 1 MB	Crash (no error returned)	Missing length check, overflow risk
message with %00 (null byte)	Output shorter than input	String terminator mishandled
recipientId 7 or 9 digits, or letters	ERR_ID	Exactly 8 digits
recipientId "' OR 1=1 --"	ERR_ID	Whitelist works for ID
keySize 100, 129, 512	ERR_KEY	Allowed: 128, 192, 256

1. Inferred validation logic and decomposition

Component	Inferred rule
Data validation	1 ≤ message length ≤ 256; recipientId matches [0-9]{8}; keySize {128, 192, 256}
Encryption	Pads and encrypts the validated message; returns ciphertext
Error handling	Returns ERR_EMPTY / ERR_LEN / ERR_ID / ERR_KEY; fails to handle oversized input and null bytes

2. Reconstructed BVA test set

TC	Field	Value	Class	Expected
1	message length	0	Below min	ERR_EMPTY
2	message length	1	Min	Encrypted
3	message length	2	Min+	Encrypted
4	message length	255	Max−	Encrypted
5	message length	256	Max	Encrypted
6	message length	257	Max+	ERR_LEN
7	message length	1,048,576 (1 MB)	Far above	ERR_LEN, no crash
8	message	contains null byte	Special	Full message encrypted or rejected, never truncated
9	recipientId digits	7	Below	ERR_ID
10	recipientId digits	8	Valid	Accepted
11	recipientId digits	9	Above	ERR_ID
12	recipientId	"12345678'; DROP TABLE x;--"	Injection	ERR_ID
13	keySize	127 / 128 / 129	Around 128	ERR_KEY / OK / ERR_KEY
14	keySize	192 and 256	Valid	OK
15	keySize	257	Above max	ERR_KEY

3. How the design prevents buffer overflow and injection

- **Check before copy:** the length is validated (1 to 256) before any buffer is filled, so test 7 must return ERR_LEN instead of crashing; the buffer is sized length + padding.
- **Boundary tests at 255/256/257** detect off-by-one writes, the classic cause of overflows.
- **Whitelist validation** (digits only for recipientId) rejects injection strings before they reach any query or command; messages are treated as data and encoded, never interpreted.
- **Fuzzing is repeated** on every release to confirm that no input outside the boundaries causes a crash.

Interpretation: the fuzz run exposed three unhandled cases (oversized input, null byte, no error on crash), and the 15 BVA tests lock the corrected boundaries in place.

Question 8: Insurance Premium Calculator – Multi-Condition Defects

Assumed logic (base premium 10,000):

```
1: premium = base
2: if (age < 25 || claims > 2) premium *= 1.5
4: if (smoker && age > 40) premium += 2000
6: if (hasDiscount && claims == 0) premium *= 0.9
8: return premium
```

1. Why the legacy suite reached 100% statement coverage but missed defects

Two legacy tests are enough for 100% statement coverage: **T1** (age 45, claims 3, smoker, no discount) and **T2** (age 22, claims 0, non-smoker, discount). Statement coverage only asks that each line runs; it does not check *which condition* made a decision true.

Seeded defect (mutant)	T1 result	T2 result	Detected?
smoker || age > 40 instead of &&	same (both true)	same (both false)	No
hasDiscount || claims == 0 instead of &&	same (both false)	same (both true)	No
Branch outcomes: decision 2 never false	—	—	Branch coverage only 5 of 6 (83%)

The inadequate criterion is **statement coverage**; the defects are in the individual conditions inside compound decisions.

2. New control flow graph

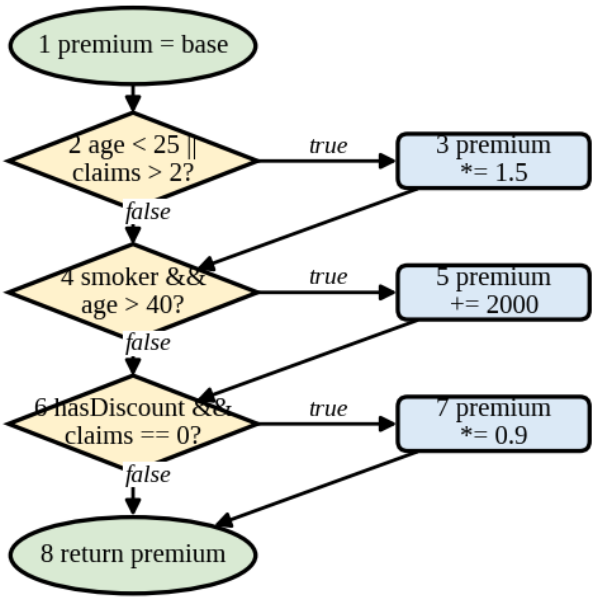


Figure 4: CFG of the premium calculator

E = 10, N = 8, V(G) = 10 − 8 + 2 = 4 (3 decisions + 1). **Four independent paths:** P1 1-2-4-6-8; P2 1-2-3-4-6-8; P3 1-2-4-5-6-8; P4 1-2-4-6-7-8. Because the three decisions are sequential and independent, there are 2^3 = 8 complete paths.

3. Path testing strategy achieving 100% branch and path coverage

TC	Age	Claims	Smoker	Discount	Decisions (2, 4, 6)	Expected premium
1	30	1	N	N	F F F	10,000

TC	Age	Claims	Smoker	Discount	Decisions (2, 4, 6)	Expected premium
2	22	1	N	N	T F F	15,000
3	45	1	Y	N	F T F	12,000
4	30	0	N	Y	F F T	9,000
5	45	3	Y	N	T T F	17,000
6	22	0	N	Y	T F T	13,500
7	45	0	Y	Y	F T T	10,800

Infeasible path: the 8th path (T T T) needs age or claims to make decision 2 true (claims > 2 since age > 40), while decision 6 needs claims = 0, which contradicts it. So 7 of 8 paths are feasible; tests 1 to 7 give 100% branch and 100% feasible-path coverage.

Condition-level extension (to kill the mutants above): add four tests with expected premium 10,000: (45, 1, non-smoker, no discount) detects smoker || age > 40; (30, 1, smoker, no discount) detects the same mutant from the other side; (30, 0, non-smoker, no discount) and (30, 1, non-smoker, discount) detect hasDiscount || claims == 0. Total = 11 tests.

Risk reduction: a wrong || would give every discount-flagged policy a 10% cut, a loss of 1,000 on each 10,000 premium; the condition-level tests make that error visible before release. **Interpretation:** statement coverage was necessary but weak; branch, path and condition tests together expose calculation errors.

Question 9: Cloud File Synchronization – Lost Files

Assumed defect reports (lost files):

Report	File size	File type	Network speed	Outcome
R1	2.0 GB and above	video	10 Mbps	Upload stops, local copy removed
R2	0 bytes	text	50 Mbps	File skipped, then deleted
R3	600 MB	archive	256 kbps	Timeout, partial file left
R4	5 MB	temp file (.tmp)	10 Mbps	Treated as system file, not synced
R5	300 MB	image set	Drops to 0 mid-upload	Upload not resumed, file lost

1. Inferred synchronization logic and developers' ECP

Factor	Likely classes tested by developers	Classes never tested
Size	Small (1 KB to 100 MB)	0 bytes; 100 MB to 2 GB; ≥ 2 GB (32-bit limit 2,147,483,647 bytes)
Type	Documents, images	Temporary/system files, archives, very long file names
Network	Fast (≥ 10 Mbps)	Slow (< 1 Mbps), intermittent, offline during upload

The defects cluster beyond the sizes and speeds that the developers' ECP covered, and the failures involve a fixed time-out and a missing resume step. **Example:** 2 GiB at 1 Mbps needs 17.18 Gbit / 1 Mbps = about 17,180 s (4.8 h), so a single whole-file time-out of 30 s cannot work; chunked upload with resume is required.

2. Revised strategy: ECP + BVA + stress testing

Technique	Test cases	Expected result
BVA on size	0 B, 1 B, 100 MB − 1, 100 MB, 2 GB − 1 B, 2 GB, 2 GB + 1 B	All synced and checksums match (0 B synced as empty file)
ECP on type	Document, image, archive, .tmp, name with 255 and 256 characters	Synced per policy; unsupported types reported, not dropped
BVA on network speed	Throttle to 10 Mbps, 1 Mbps, 256 kbps, 64 kbps, then disconnect	Upload resumes from last chunk; no data loss
Combination (size x speed)	2 GB at 256 kbps; 600 MB at 64 kbps	Completes over time with resume
Stress	1,000 concurrent clients; 10,000 small files; 24 h soak	No lost or duplicated file; memory stable
Failure injection	Kill network or process at 10%, 50%, 99% of a transfer	Local file kept until server acknowledges

3. How the approach prevents future data loss

- **Boundary tests** remove the 2 GB and 0-byte failures; **chunking with resume** removes the time-out failures.
- **Checksum verification** before deleting the local copy, so a failed upload can never destroy the only copy.
- **Stress and failure-injection tests** run on every release, so loss caused by load or network drops is caught before users see it.

Interpretation: every one of the five reported losses maps to a test class in the revised strategy, so each would now be detected.

Question 10: E-commerce Search and Filter – Decision Table

Assumed SQL sequence:

```
Q1: ... WHERE category='Shoes' AND price BETWEEN 1000 AND 3000 AND in_stock=1
Q2: ... WHERE category='Shoes' AND brand IN ('Nike','Puma') AND NOT on_sale=1
Q3: ... WHERE price BETWEEN 1000 AND 3000 AND brand IN ('Nike') AND in_stock=1
Q4: ... (no WHERE) when no filter is selected
```

1. Inferred decision logic

Observation	Inferred logic
Different filters (category, price, stock) are always joined by AND	Each selected filter narrows the result (AND across filters)
Several brands appear in one IN (...) list	OR within the same filter (multiple values of one attribute)
on_sale handled as NOT on_sale=1	NOT used for exclusion filters
No filter selected gives no WHERE clause	Empty condition set returns all products

2. Decision table (4 filters: Category, Brand, Price, In-stock only)

Rule	Category	Brand	Price	In stock	Expected WHERE clause
1	N	N	N	N	(none): all products
2	N	N	N	Y	stock
3	N	N	Y	N	price
4	N	N	Y	Y	price AND stock
5	N	Y	N	N	brand
6	N	Y	N	Y	brand AND stock
7	N	Y	Y	N	brand AND price
8	N	Y	Y	Y	brand AND price AND stock
9	Y	N	N	N	category
10	Y	N	N	Y	category AND stock
11	Y	N	Y	N	category AND price
12	Y	N	Y	Y	category AND price AND stock
13	Y	Y	N	N	category AND brand
14	Y	Y	N	Y	category AND brand AND stock
15	Y	Y	Y	N	category AND brand AND price
16	Y	Y	Y	Y	category AND brand AND price AND stock

3. Additional partitions to complete the model

Dimension	Partitions	Test values
Brand values	One brand / two or more brands (OR)	Nike; Nike + Puma
Price range	Valid / min > max (invalid) / boundary equal to a product price	1000 to 3000; 3000 to 1000; 1000 to 1000
Exclusion	NOT on_sale on / off	on_sale excluded / included
Result size	Some results / zero results	Filter combination with no match

Dimension	Partitions	Test values
Security	Injection in a text filter	' OR 1=1 -- as brand text

4. Completeness evaluation

With 4 binary filters there are $2^4 = 16$ combinations, and the table lists all 16 (100% of on/off combinations). If a fifth filter (rating ≥ 4) is added the table doubles to 32 rules, which shows why value-level partitions (above) are added selectively rather than multiplied. **Limits of the model:** it checks which filters are active, not value correctness, sorting or pagination, so BVA on price and result-set checks are added as separate test cases. **Expected test count:** 16 combination tests + 5 partition tests + 1 injection test = 22.

Interpretation: the decision table turns the inferred AND/OR/NOT logic into a complete, checkable set of combinations, and the additional partitions cover what the table cannot.